THE UNIVERSITY OF SYDNEY

School of Computer Science
Faculty of Engineering
Semester 2, 2026

COMP9001 Introduction to Programming

Assignment 2 - Civilisation Simulator

Marks	20
Due	11:59pm Sunday 25th October 2026 (3 weeks)
Submit	one zip file on Canvas containing your report (PDF) and all your `.py` files
Provided	`events.txt`, `stages.txt`, `rules.txt`, `sample_civilisation_history.txt`
Topics	conditions, loops, functions, lists and dictionaries, classes, break/continue, return values, exceptions, text-file input and output

Overview

You will write a Python program that simulates a civilisation growing from an **Ancient Settlement** into a **Space Colony**. The rules of the world (natural disasters, growth, costs, stage requirements) are *fixed* and are read from three text files that we provide. Your job is to:

1. implement the simulation rules exactly as described in this document;
2. design an automatic **investment strategy** that helps your civilisation advance as far as possible;
3. design and implement one **civilisation trait** unique to you;
4. record the civilisation's history in a text file; and
5. write a short report explaining your design and results.

Because the rules are the same for everyone, the *quality of your strategy* decides how far your civilisation gets. Poor strategies often stall in the middle ages or starve; a careful one can reach a Space Colony in a shorter amount of years.

1 Timeline

The simulation runs from **Year 0 to Year 3000**. It does not simulate every year: one **turn** represents **10 years**.

- **Year 0** is the starting state. It is recorded in the history file, but it is *not* a turn (no event, no investment).
- Turns happen in Years **10, 20, 30, . . . , 3000**, so there are at most **300 turns**.
- The simulation can end earlier (see Step 7 in Section 5).

2 Resources

Your civilisation has five whole-number resources: **Population** (produces food and taxes, consumes food), **Food**, **Treasury** (pays for investments), **Infrastructure** (sets how many people the land can support) and **Technology** (increases food production). Population, Infrastructure and Technology are also needed to advance. Whenever a rule changes a resource, Population, Food and Treasury never go below 0, and Infrastructure and Technology never go below 1.

Whole numbers. Every amount you calculate is turned into a whole number by *dropping its decimal part*: 710.7 becomes 710 and −710.7 becomes −710. In this document $\lfloor x \rfloor$ means "x with its decimal part dropped".

3 Provided data files

Your program must **read** these three files when it starts; do not copy their values into your program or edit them. Blank lines and lines starting with `#` are comments.

- `rules.txt`: every constant, one per line as `name = value` (e.g. `growth_rate = 0.08`). Names in `typewriter font` in this document refer to this file; the number in brackets is its value.
- `events.txt`: one event per line: name, lower bound, upper bound, effects. Effects are separated by `;`, and a change ending in `%` is a percentage, otherwise a fixed amount, e.g. `Flood, 0.45, 0.59, Food:-20%; Infrastructure:-1`.
- `stages.txt`: one stage per line: number, name, minimum population, minimum infrastructure, minimum technology.

Use **exception handling** so that a missing file prints a clear message and stops the program (no Python error trace), and a line that cannot be understood is skipped with a warning.

4 Starting conditions

When the program starts it asks the user for a **9-digit Student ID**. If the input is not exactly 9 digits, it prints a message and asks again.

Write the digits as `ABCDEFGHI`. `ABC` means the *number* formed by the first three digits (so `042` is 42).

Resource	Starting value
Population	$800 + \texttt{ABC} \times 2$
Food	$2000 + \texttt{DEF} \times 3$
Treasury	$1000 + \texttt{GHI} \times 2$
Infrastructure	$1 + (\texttt{A} + \texttt{D} + \texttt{G}) \bmod 4$
Technology	$1 + (\texttt{C} + \texttt{F} + \texttt{I}) \bmod 3$

Example: Student ID `500123456` starts with Population 1,800, Food 2,369, Treasury 1,912, Infrastructure 3, Technology 1.

Random numbers. Immediately after reading the Student ID, **seed** Python's random number generator **once** with the Student ID (as a whole number). Your program then produces the *same* history every time it is run with the same ID. Markers will re-run your program with your ID, and the output must match the `civilisation_history.txt` you submit.
Your program uses exactly two kinds of random number, both made with the `random` module's *randint* function: the **event number** (Step 2) and the **advancement roll** (Step 5). Do not generate any other random numbers.

5 One turn

Every turn performs these steps **in this order**:

1 Update resources → 2 Event → 3 Choose investment → 4 Apply investment → 5 Advancement → 6 Record & display → 7 End checks

Step 1 – Update resources

Let P, F, R, I and T be the current Population, Food, Treasury, Infrastructure and Technology. Carry out the following **in order**; each part uses the values produced by the parts before it.

1a. Food.

$$\text{production} = \lfloor P \times (\texttt{food_base_production} + \texttt{food_production_per_technology} \times T) \rfloor$$
$$\text{consumption} = \lfloor P \times \texttt{food_per_person} \rfloor$$
$$F \leftarrow F + \text{production} - \text{consumption}$$

1b. Storage limit. Food cannot be stored for more than `food_storage_turns` turns: if F is greater than $\lfloor P \times \texttt{food_storage_turns} \rfloor$, set F to that value.

1c. Population. Let capacity $= I \times \texttt{capacity_per_infrastructure}$. Exactly one of these applies:

- *Famine* – if F is negative: set F to 0 and $P \leftarrow \lfloor P \times (1 - \texttt{famine_loss}) \rfloor$.
- *Overcrowding* – otherwise, if P is greater than capacity:
  $P \leftarrow \lfloor P \times (1 - \texttt{overcrowding_loss}) \rfloor$, but not below capacity.
- *Growth* – otherwise: $P \leftarrow \lfloor P \times (1 + \texttt{growth_rate}) \rfloor$, but not above capacity.

1d. Treasury. Using the *new* population:

$$\text{income} = \lfloor P \times \texttt{tax_per_person} \rfloor, \qquad \text{upkeep} = I \times \texttt{upkeep_per_infrastructure}$$
$$R \leftarrow R + \text{income} - \text{upkeep} \quad \text{(not below 0)}$$

Constant	Value	Constant	Value
`food_base_production`	0.75	`famine_loss`	0.20
`food_production_per_technology`	0.05	`overcrowding_loss`	0.10
`food_per_person`	1.00	`growth_rate`	0.08
`food_storage_turns`	5	`tax_per_person`	0.15
`capacity_per_infrastructure`	3000	`upkeep_per_infrastructure`	40

Step 2 – Natural event

Generate a random whole number from 0 to 99 (inclusive) and divide it by 100 to get the **event number** (0.00 to 0.99). The event that happens is the one in `events.txt` whose lower and upper bounds contain the event number (bounds included). If no event matches (for example because a bad line was skipped), treat it as *No Major Event*.

Number	Event	Effects (from `events.txt`)
0.00–0.14	Drought	Food −30%
0.15–0.29	Disease	Population −15%
0.30–0.44	Invasion	Population −10%, Treasury −20%
0.45–0.59	Flood	Food −20%, Infrastructure −1
0.60–0.69	Earthquake	Infrastructure −2, Treasury −15%
0.70–0.79	Excellent Harvest	Food +30%
0.80–0.89	Trade Boom	Treasury +30%
0.90–0.94	Scientific Breakthrough	Technology +1
0.95–0.99	No Major Event	none

A percentage change is $\lfloor \text{current value} \times \text{percentage}/100 \rfloor$. For example, a Drought when Food is 2,369 changes Food by $\lfloor 2369 \times (-30)/100 \rfloor = -710$. An event is **damaging** if at least one of its effects is negative.

Step 3 – Choose an investment (your strategy)

Each turn the program automatically chooses **at most one** resource to invest in. A resource is **available** only if both of these are true:

- **Cooldown:** it was not chosen in either of the previous `cooldown_turns` (2) turns. If Technology is chosen in Year 100, it is unavailable in Years 110 and 120 and available again in Year 130.
- **Affordable:** its cost (see Step 4) is not more than the current Treasury.

For every available resource your program calculates a **priority score**

$$\text{score} = \text{Base Priority} + \text{Resource Need} + \text{Advancement Need} + \text{Event Effect}$$

and chooses the resource with the **highest** score. If two scores are equal, choose the one that comes first in the order Population, Food, Treasury, Infrastructure, Technology. If no resource is available, there is no investment this turn (record it as `None`).

You decide how each part of the score is calculated. Your strategy should take into account:

- **Base Priority** – how important each resource is in general;
- **Resource Need** – whether a resource is low or critically low (e.g. Food that will not last another turn, or Population close to capacity);
- **Advancement Need** – whether the resource is below the next stage's requirement;
- **Event Effect** – the damage this turn's event caused.

Illustration only (the numbers are not a recommended strategy). After an Invasion, a civilisation needs Technology 8 for the next stage but has 7:

	Base	Resource need	Advancement need	Event effect	Score
Population	10	0	0	15	25
Treasury	10	0	–	15	25
Technology	15	0	25	0	**40**

Technology is chosen, even though the Invasion damaged Population and Treasury.

Step 4 – Apply the investment

Work out the cost and the gain from the resource values *before* the investment. Take the cost from Treasury, then add the gain. The numbers in the table are the values of `<resource>_investment_gain` and `<resource>_investment_cost` in `rules.txt` (for example `food_investment_gain`); read them from the file.

Investment	Gain	Cost (taken from Treasury)
Population	$\lfloor P \times 0.10 \rfloor$ people	$\lfloor P \times 0.20 \rfloor$
Food	$\lfloor P \times 0.80 \rfloor$ food	$\lfloor P \times 0.10 \rfloor$
Treasury	$\lfloor P \times 0.40 \rfloor$ treasury	0
Infrastructure	+2 infrastructure	$I \times 1200$
Technology	+1 technology	$T \times 1200$

Step 5 – Advancement

There are ten stages, listed with their minimums in `stages.txt`: from Ancient Settlement (stage 1), through e.g. Empire (12,000 people, Infrastructure 16, Technology 8), to Space Colony (120,000 people, Infrastructure 60, Technology 30).

Every civilisation starts at stage 1. At most **one** stage can be gained per turn, and a civilisation never goes back a stage.

The civilisation is **eligible** to advance if it meets all three minimums of the *next* stage **and** Food is above 0 **and** Treasury is above 0. If it is not eligible, nothing happens (and no random number is generated). If it is eligible, calculate the **advancement chance** (a whole-number percentage):

Start with	`base_advancement_chance` (50)
+`advancement_bonus` (10)	if Food ≥ `comfortable_food_turns` (1.0) × consumption, where consumption is $\lfloor P \times$ `food_per_person` $\rfloor$
+10	if Treasury ≥ `comfortable_treasury_per_stage` (500) × the next stage's number
+10	if Infrastructure is *greater than* the next stage's minimum
+10	if Technology is *greater than* the next stage's minimum
−`damaging_event_penalty` (20)	if this turn's event was damaging
Maximum	`max_advancement_chance` (90)

Then generate the **advancement roll**, a random whole number from 0 to 99 (inclusive). The civilisation advances if the roll is *less than* the chance; otherwise it stays at its current stage and may try again in a later turn.

Step 6 – Record and display

Write the turn to the history file (Section 7) and update the terminal display when required (Section 8).

Step 7 – End checks

After recording the turn, the simulation **ends immediately** if any of these is true:

- Population is below `minimum_population` (250) *extinct*
- Food has been 0 at the end of `max_zero_food_turns` (2) turns in a row *extinct*
- the civilisation has reached **Space Colony** *success*
- the turn was Year 3000 *time limit*

6 Your civilisation trait

Take the last two digits of your Student ID as a number and find its remainder when divided by 5: **0** Agricultural, **1** Commercial, **2** Militaristic, **3** Scientific, **4** Engineering.

Design and **implement** one **benefit** and one **disadvantage** that suit your trait. For example, a Scientific civilisation might pay 25% less for Technology but have 10% lower food production.

- The trait must be active in the run you submit, and both effects must change what happens in the simulation.
- Put the trait in your program (for example as an extra multiplier on one of the rules). Do **not** edit the data files.
- Apart from your trait, every rule in Section 5 must be followed exactly.

7 History file

Your program must create `civilisation_history.txt` (replacing any old copy) and record the **complete state** at Year 0 and after every turn, in exactly this format. Numbers use a comma as the thousands separator.

```
Student ID: 500123456
Trait: None

YEAR 630
Stage: Kingdom
Population: 24,526
Food: 48,688
Treasury: 13,107
Infrastructure: 11
Technology: 6
Event: Trade Boom
Investment: Infrastructure
```

```
Advancement: ADVANCED to Kingdom (chance 80%)
...
SIMULATION ENDED IN YEAR 2550: SPACE COLONY REACHED
```

The `Advancement` line is one of `Starting state` (Year 0), `Requirements not met`, `Failed (chance N%)` or `ADVANCED to <stage> (chance N%)`. A complete example (Student ID `500123456`, without a trait) is in `sample_civilisation_history.txt`.

8 Terminal display

Printing all 300 turns is too much. Show a summary panel of the civilisation:

- at Year 0;
- every 100 years (every 10th turn);
- on every turn in which the civilisation advances; and
- a final summary when the simulation ends (final year, final stage, reason).

For example:

```
+============================================================+
| YEAR 1000                                                  |
| INDUSTRIAL NATION                                          |
+------------------------------------------------------------+
| Population                                        51,000   |
| Food                                             280,500   |
| Treasury                                          16,471   |
| Infrastructure                                        22   |
| Technology                                            15   |
+------------------------------------------------------------+
| Event: Disease                                             |
| Investment: Infrastructure                                 |
| Advancement: ADVANCED to Industrial Nation (chance 60%)    |
+============================================================+
```

You may design your own layout, symbols, progress bars, colours or emoji; columns should line up neatly. Some emoji are displayed two characters wide, so boxes that contain them may not line up perfectly – that is fine.

9 Program requirements

Your program should make good use of the Python content covered in the unit, including:

- **classes and objects**, with responsibilities split sensibly between them, including at least one **class variable** and one **static or class method**;
- **functions and methods** that take parameters and return values;
- **lists and dictionaries** to organise your data;
- **conditions and loops** to control the simulation;
- **file input and output** for the data files and the history file;
- **exception handling** for invalid input and problems with the files;

- **formatted output** for the terminal display and the history file.

Restrictions:

- Only import `random`, `sys` and your own modules.
- Do not use recursion.
- Your program must run from start to finish without crashing for any valid Student ID.

10 Report

Your report should be about **5 pages (recommended)** and must be **at most 7 pages**, not counting the code appendix. The results you describe must come from running your program with your own Student ID.

Q1 – Strategy, trait and results (suggested length 1 page)

Explain how your priority score is calculated and why you chose that design, and describe your trait's benefit and disadvantage and where they are implemented. Using your history file, give:

- your final stage, the final year, why the simulation ended, and the years in which your civilisation advanced;
- one turn where your strategy made a good decision and one where it could have done better (quote the history records), and how you would improve it;
- evidence of both trait effects.

Q2 – Flowchart (suggested length 2 pages)

Draw one flowchart of your whole program: loading the data files, starting conditions, the seven steps of a turn, and all four end conditions. Show where the loop repeats and where it stops. Hand-drawn (scanned) or software-drawn are both acceptable, but it must be readable.

Q3 – Class design (suggested length 1-2 pages)

Describe each class: what it is responsible for, its important variables and methods, and how the objects communicate. You may use a mixture of text and visual explanations such as a class diagram. Explain why you put at least two responsibilities into separate classes instead of one large class.

Q4 – Error handling and user experience (suggested length 1 page)

Explain how your program copes when things go wrong and how it helps the person using it:

- invalid input (e.g. a Student ID that is not 9 digits) and what the user is told;
- problems with the data files (a missing file, a badly formatted line), with a short example of your exception handling;
- how your terminal display and messages make the simulation easy to follow.

Include at least one piece of terminal output showing an error being handled.

Appendix – Source code

Include all your `.py` files, as text or screenshots, identical to the code in your zip file.

11 Submission

Submit **one zip file** on Canvas containing:

- your report as a PDF, including the code appendix; and
- all your `.py` files, so that the program runs with `python3 civilisation.py`.

You do not need to include the three data files; markers use the original ones. Running your code with your Student ID must reproduce the results in your report.

You can use the Assignment 2 workspace on Ed to write and test your code (it is not auto-graded), but only the zip file on Canvas is marked. The code must match the appendix of your PDF, and running it with your Student ID must reproduce the results in your report.

Late submissions. Unit Outline

Academic integrity and generative AI. Unit Outline

Marking. The assignment is marked out of **20**. Markers run the code from your zip file with your Student ID and two other test IDs, and try an invalid Student ID and a missing or broken data file.

Criterion	Marks	What is assessed
Flowchart (Q2)	4	A complete, readable flowchart of the whole program: set-up, the seven steps of a turn, the loop and all four end conditions.
Program runs robustly	5	Your code runs to the end without crashing for your Student ID and other IDs, follows the specification, writes the history file correctly and reproduces the results in your report.
Code structure and design	4	Clear, well-organised classes and methods, the required Python features, readable names and useful comments.
Strategy, trait and results (Q1)	4	A clear explanation of your priority score and trait, supported by evidence from your own run.
Error handling and user experience (Q4)	3	Sensible handling of invalid input and data-file problems, with clear messages and an easy-to-follow display.
Total	**20**	